Cheatsheet · 15 chapters on one page

Manual Testing cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

146 key ideas | 19 formulas | 166 tables | 87 tips and traps

STEP 1 Foundations

1.1 Software Testing Fundamentals

A bank asks a software company for an application. Developers write the code, test engineers check it against the requirement, and bugs travel back and forth until the product is stable. This page explains that loop and every word you need to talk about it.

KEY IDEAS

- **What software is and who builds it** Software is a business need of a customer, turned into a working program. A bank such as ICICI wants its customers to log in, check a balance, transfer money and apply for a loan. The bank is not a software company, so it hands this need to one.
- **What testing means** There are three common definitions, and a good answer in an interview mentions all three:
- **Manual testing** Manual testing is the process of testing an application as per the client requirement, done by a test engineer by hand, without any tool running the steps.
- **Automation testing** Automation testing is testing an application with the help of a tool. A tester writes or records a test script once, and the tool runs it again and again.
- **Programming and scripting languages**
- **White box, black box and gray box** Testing is grouped by how much of the inside the tester can see.
- **The seven principles of testing** These seven principles (as taught by ISTQB) explain why testers work the way they do.
- **Why a late bug costs more** A mistake in the requirement that is found during a requirement review costs a short conversation. The same mistake found after release means changing the design, the code, the tests and the documents, then shipping again.
- **Who does what in a software company** A tester's career usually grows from test engineer to senior test engineer, test lead and test manager, or towards automation engineer and SDET (software development engineer in test).
- **QA, QC and testing: three different jobs** People often use these words as if they mean the same thing. They do not, and interviewers love to ask about the difference.
- **Test oracles and the tester mindset** To call something a bug, a tester must know what the right result is. Whatever tells you the right result is called a test oracle . The requirement is the main one, but it is never complete, so good testers use several:

FORMULAS

The seven principles of testing Exam favourite

$$10^{10} = 10{,}000{,}000{,}000$$

Lesson: The seven principles of testing

The seven principles of testing

$$10^{20}$$

Lesson: The seven principles of testing

TABLES

Table: module · what the customer asked for

Module	What the customer asked for
Login	Username of 5 alphabets, password of 5 alphabets, an OK and a Cancel button
Account balance	Show the current balance of the logged in user
Amount transfer	Move money from one account to another
Loan	Apply for a loan and track it
Logout	End the session safely

Lesson: What software is and who builds it

Table: tool · used for · notes

Tool	Used for	Notes
QTP (now UFT One)	Functional testing	Created by Mercury, later HP, now sold by OpenText. VB Script. Licensed, comes with support
Selenium	Functional testing of web applications	Open source and free. Scripts in Java, Python, JavaScript and more
LoadRunner	Performance testing	Licensed. Simulates thousands of virtual users
JMeter	Performance testing	Open source and free
Quality Center (ALM)	Test management	Requirements, test cases, execution and defects in one place
Bugzilla, Jira, Mantis	Bug tracking	Log, assign and follow every defect

Lesson: Automation testing

Table: # · username · password · expected

#	Username	Password	Expected	Actual	Status
1	Dinga	Ranii	Home page	Home page	Pass
2	Ding	Ranii	Invalid username or password	Invalid username or password	Pass
3	Dingaa	Ranii	Invalid username or password	Home page	Fail
4	Ding4	Ranii	Invalid username or password	Invalid username or password	Pass
5	(blank)	Ranii	Username can't be blank	Username can't be blank	Pass
6	Dinga	(blank)	Password can't be blank	Password can't be blank	Pass
7	Dinga	Ran1i	Invalid username or password	Invalid username or password	Pass
8	DINGA	abcde	Home page	Home page	Pass

Lesson: Automation testing

Table: programming languages · scripting languages

Programming languages	Scripting languages
C, C++, C#, Java, .NET languages	VB Script, JavaScript, Python, Shell, Perl
Used to **develop** applications	Used to **automate, validate and test**
Usually compiled into an executable or bytecode	Usually run line by line by an interpreter
Larger programs with strict structure	Shorter programs that glue steps together
Run on servers and machines	Often run in a browser or inside another tool

Lesson: Programming and scripting languages

Table: · white box testing · black box testing · gray box testing

	White box testing	Black box testing	Gray box testing
Also called	Glass box, unit, structural testing	Functional, behavioural testing	A mix of both
Done by	Developers	Test engineers	People with coding and testing knowledge
Looks at	The code: every line, path, condition and loop	The application: inputs, outputs and behaviour	Behaviour, with some knowledge of the inside
Code is	Visible	Invisible	Partly visible (for example the database)
Needs	Programming knowledge	Application and domain knowledge	Both

Lesson: White box, black box and gray box

Table: principle · what it means in practice

Principle	What it means in practice
1. Testing shows the presence of defects	Testing can prove that bugs exist. It can never prove that none are left.
2. Exhaustive testing is impossible	Every possible input is far too many to try, so we choose values smartly (boundary values, equivalence classes).
3. Early testing saves time and money	Reviewing the requirement finds mistakes before any code is written.
4. Defects cluster together	A small number of modules usually contains most of the bugs.
5. The pesticide paradox	Running the same tests forever stops finding new bugs. Tests must be reviewed and extended.
6. Testing is context dependent	A banking app, a game and a heart monitor are tested very differently.
7. Absence of errors is a fallacy	A bug free app that does not meet the user's need is still a failure.

Lesson: The seven principles of testing

Table: module · bugs · running total

Module	Bugs	Running total
Compose	48	48%
Chat	32	80%
Settings	7	87%
Inbox	5	92%
Login	3	95%
Sent items	2	97%
Drafts	1	98%
Trash	1	99%
Help	1	100%
Logout	0	100%

Lesson: The seven principles of testing

Table: phase where the bug is found · relative cost to fix

Phase where the bug is found	Relative cost to fix
Requirement	1×
Design	5×
Coding	10×
Testing	20×
After release	100×

Lesson: Why a late bug costs more

Table: phase found · cost per bug · late team · early team

Phase found	Cost per bug	Late team	Early team
Requirement	1	2	15
Design	5	3	10
Coding	10	10	15
Testing	20	25	8
After release	100	10	2
Total cost		**1617**	**575**

Lesson: Why a late bug costs more

Table: role · main job

Role	Main job
Business analyst (BA)	Collects the requirement from the customer and writes it down (service based companies)
Product analyst (PA)	Studies the market and decides what the product needs (product based companies)
Architect	Designs the high level structure: the modules and how they connect
Development manager	Plans the low level design and leads the developers
Developer	Writes the code and tests it with white box testing
Test engineer	Writes test documents, executes tests, reports and retests bugs
Test lead and test manager	Plan the testing, assign modules, review work, report quality
Build and release team	Turn the code into a build and install it on the servers
Customer	Gives the requirement and finally accepts the product (UAT)

Lesson: Who does what in a software company

Table: · quality assurance (qa) · quality control (qc) · testing

	Quality assurance (QA)	Quality control (QC)	Testing
Focus	The **process**: are we working in a way that prevents defects?	The **product**: does this product meet the standard?	A main activity inside QC: running and checking the product
Goal	Prevent defects	Find defects	Find defects and give information about quality
When	All through the project	When there is something to inspect	Whenever something can be checked
Examples	Review templates, coding standards, training, audits, retrospectives	Reviews of the product, testing, inspections, sign off	Functional, integration, system and acceptance testing
Owned by	Everyone, led by managers and the QA function	Testers and reviewers	Testers, and developers at unit level

Lesson: QA, QC and testing: three different jobs

Table: word · meaning · example

Word	Meaning	Example
Error (mistake)	A human action that produces a wrong result	The developer reads "greater than 5,000" as "5,000 or more"
Defect (bug, fault)	The flaw that the error leaves in a work product	The code says amount >= 5000 instead of amount > 5000
Failure	What the user sees when the defect is executed	A transfer of exactly 5,000 goes through when it should be blocked
Root cause	The deepest reason the error happened	The requirement used the word "above" without saying whether 5,000 is included

Lesson: QA, QC and testing: three different jobs

Table: oracle · example question

Oracle	Example question
The requirement, user story or acceptance criteria	What does SRS 4.2 say should happen?
A previous version of the product	Did the old app round this amount the same way?
A comparable product	How do other banking apps show a failed transfer?
Standards and laws	Does this meet WCAG contrast rules, or the RBI rule on OTP?
A domain expert or the product owner	Should a holiday count as a working day for this rule?
Consistency inside the product	Every other page says "Sign in". Why does this one say "Login"?
User expectations and common sense	Would a real customer understand this message?

Lesson: Test oracles and the tester mindset

TIPS AND TRAPS

REMEMBER — Testing never stops at "it works for me". The question is always: **does it behave exactly as the customer's requirement says?**

Lesson: What software is and who builds it

REMEMBER — Notice how one small requirement, "username of 5 alphabets", already needs many test values: exactly 5 letters, 4 letters, 6 letters, digits, a space, a blank field, copy and paste. Thinking of these values is the core skill of a tester.

Lesson: What testing means

REMEMBER — The line between the two has blurred. Python and JavaScript now build whole products, and Java is used to write Selenium test scripts. What matters is the job: building the product, or checking it.

Lesson: Programming and scripting languages

IN THE EXAM — Interviewers ask about black box testing more than anything else, because that is the daily work of a test engineer. The next pages go deep into white box testing and then into every level of black box testing.

Lesson: White box, black box and gray box

REMEMBER — This is the idea behind "shift left": involve testers from the very first phase, review every document, and catch defects where they are cheapest.

Lesson: Why a late bug costs more

REMEMBER — Not every defect causes a failure. A defect in a screen nobody opens stays silent. And not every failure comes from a defect: a power cut or radiation can also cause one. That is why testers report failures and developers look for defects.

Lesson: QA, QC and testing: three different jobs

REMEMBER When oracles disagree, for example the requirement says one thing and the old version did another, do not guess. Raise it as a question. Many important bugs are found exactly in these gaps.

Lesson: Test oracles and the tester mindset

1.2 SDLC and its Models

Building software is like building a house: collect the need, check it is possible, design, build, test, hand over and maintain. The order is fixed, but how you walk through it is not. Each walk is called a model, and choosing the right one decides cost, speed and quality.

KEY IDEAS

- **What SDLC is: build it like a house** SDLC (Software Development Life Cycle) is the step by step procedure used to develop software. The notes compare it with building a house, because the steps line up almost exactly:
- **Service based and product based companies**
- **The seven phases, one by one** The HLD splits the application into modules . For Gmail: login, compose, inbox, sent items, outbox and logout. The LLD describes the objects inside each module. For the login page: the username box, the password box, the OK button and the Cancel button.
- **CRS, SRS and what makes a good requirement**
- **The waterfall model** The waterfall model is the basic model from which all the others grew. The phases run strictly one after another , and the output of one phase is the input of the next, like water falling down steps. It never flows back up.
- **The spiral model** Waterfall's problem: an Excel like product with four modules (A: cells, B: operations, C: merging cells, D: macros), six months each, reaches the customer only after two years .
- **The prototype model** The notes tell of a girl abroad who sees a photo of the boy before agreeing to meet him. The photo is a prototype : a sample that looks about 90% like the real thing.
- **The V and V model (V model)** The V model pairs every document on the left with a level of testing on the right. While the left side is being written, testers review each document (verification) and prepare the test document for the matching level.
- **Agile and Scrum** Agile is used in most companies today. The idea: build software quickly in small pieces, with the customer involved all the time . The notes show why.
- **Hybrid models and choosing a model** A hybrid model combines two models to get the strengths of both. The usual pairs are spiral plus prototype (each module gets a prototype approved before it is built, module by module) and prototype plus V (a prototype is approved, then the V model's reviews ...
- **Kanban, scaled Agile and DevOps** Scrum is not the only Agile way of working. Three more names appear in almost every job description today.
- **Where testing happens in each model** Every model tests, but at different moments and in different amounts. Knowing this helps a tester plan, and it is a common interview question.

FORMULAS

The spiral model Exam favourite

$$T(A), \text{ then } T(B) + T(A), \text{ then } T(C) + T(A + B), \text{ then } T(D) + T(A + B + C)$$

Lesson: The spiral model

Agile and Scrum Exam favourite

$$250 \div 15 \approx 16.7$$

Lesson: Agile and Scrum

TABLES

Table: building a house · building software

Building a house	Building software
The owner tells the builder the budget, the number of rooms and the deadline	**Requirement collection**: the business analyst writes down what the customer needs
The builder checks time, cost and labour	**Feasibility study**: is it possible with our time, money, people and technology?
An architect draws the room layout, then the wall widths	**Design**: high level design (modules), then low level design (objects)
Masons build the house	**Coding**: developers write the code
An inspector checks doors, taps and wiring	**Testing**: test engineers check the application
House warming: keys are handed over	**Installation**: the software is deployed for the customer
Leaks and cracks are repaired later	**Maintenance**: problems found by the customer are fixed

Lesson: What SDLC is: build it like a house

Table: phase · done by · what happens · output

Phase	Done by	What happens	Output
1. Requirement collection	BA or PA	Business needs are collected and written down properly	Requirement document (CRS, then SRS)
2. Feasibility study	Finance, HR, architect, project manager	Check time, cost, resources and technology: is the project doable?	Go or no go decision
3. Design	Architect (HLD), development manager (LLD)	The blueprint: modules first, then objects inside each module	Design document
4. Coding	Developers	Code is written from the design, and checked with white box testing	Source code, builds
5. Testing	Test engineers	The application is tested and bugs are reported, fixed and retested until it is stable	Test results, bug reports
6. Installation (deployment, rollout)	Installation engineers	The stable software is installed for the customer	Live application
7. Maintenance	Maintenance engineers (for example one developer and one tester)	Problems found by the customer are fixed, retested and delivered quickly	Patches, updates

Lesson: The seven phases, one by one

Table: quality · bad example · good example

Quality	Bad example	Good example
Detailed	"Login should work"	"Username accepts exactly 3 digits"
Easy to understand	Long sentences with unclear terms	Short numbered points in simple words
Proper flow	Modules listed in random order	Login, then home, then compose, in the order a user moves
Measurable	"The page should load fast"	"The home page loads within 3 seconds for 1,000 users"
Testable and unambiguous	"Amount should be reasonable"	"Amount from 5,000 to 50,000, whole numbers only"

Lesson: CRS, SRS and what makes a good requirement

Table: phase · waterfall · v model

Phase	Waterfall	V model
Requirement	1 to 4	1 to 4
Feasibility	5 to 6	5 to 6
Design	7 to 12	7 to 12
Coding	13 to 24	13 to 24
Testing	25 to 32	25 to 29
Installation	33	30

Lesson: The waterfall model

Table: advantages · disadvantages

Advantages	Disadvantages
The customer uses part of the software early. The model is flexible for changes.	No parallel deliverables. No requirement review. The customer may still reject the look or behaviour at the end, a drawback shared with waterfall.

Lesson: The spiral model

Table: cycle · delivered · month · tested in this cycle

Cycle	Delivered	Month	Tested in this cycle	Module tests so far
1	Module A	6	A	1
2	Module B	12	A and B	3
3	Module C	18	A, B and C	6
4	Module D	24	A, B, C and D	10

Lesson: The spiral model

Table: document (left side) · written by · testers review it and prepare · executed as (right side)

Document (left side)	Written by	Testers review it and prepare	Executed as (right side)	Executed by
CRS	BA	Acceptance test document (ATD)	User acceptance testing	Customer
SRS	BA	System test document (STD)	System testing	Test engineers
HLD	Architect	Integration test document (ITD)	Integration testing	Test engineers
LLD	Development manager	Functional test document (FTD)	Functional testing	Test engineers
Coding	Developers	Unit test programs	White box testing	Developers

Lesson: The V and V model (V model)

Table: term · meaning

Term	Meaning
Scrum team	7 ± 2 people, for example 2 designers, 2 developers, 2 testers and the scrum master
Scrum master	Leads the process, removes blockers, picks stories for each sprint with the team
Product owner	The person or organisation who gives the requirements and sets their priority
Pigs and chickens	From an old fable: pigs are committed members who do the work (designers, developers, testers). Chickens are involved but not committed (managers and stakeholders)
Product backlog	Every requirement from the customer, for example all 50 user stories
Sprint backlog	The stories chosen for one sprint, for example 10 of the 50
Sprint	A fixed time box (often 10 to 15 working days) to build and test the sprint backlog
Carry forward	A story not finished in a sprint moves to the next sprint
Release	Handing over the stable increment to the customer

Lesson: Agile and Scrum

Table: meeting · when · who and what

Meeting	When	Who and what
Project kick off	Before the project starts	BA, scrum master and senior people agree on the goals
Sprint planning	Before every sprint	The team and scrum master choose the sprint backlog
Daily stand up	Every day, about 15 minutes	What I did yesterday, what I will do today, what is blocking me
Sprint review	End of every sprint	The increment is shown to the product owner
Retrospective (post mortem)	After a sprint or release	What went wrong, what went well, what to improve

Lesson: Agile and Scrum

Table: sprint · points planned · finished · carried forward

Sprint	Points planned	Finished	Carried forward
1	13	Login	Register
2	16	Register	Compose
3	16	Compose, Inbox	Sent items
4	16	Sent items, Drafts, Trash, Search	
5	6	Settings, Logout	

Lesson: Agile and Scrum

Table: model · changes allowed · parallel work · requirement review

Model	Changes allowed	Parallel work	Requirement review	Best for
Waterfall	No	No	No	Small, fixed, life or machine critical projects
Spiral	Yes	No	No	Dependent modules, delivered one by one
Prototype	Yes	No	No (customer reviews the dummy)	Customers new to the application or unclear about it
V model	Yes, with rework	Yes	Yes	Large and complex applications
Hybrid	Yes	Depends on the pair	Depends on the pair	Projects that need two strengths at once
Agile (Scrum)	Yes, every sprint	Yes	Continuous, with the customer	Most modern projects with changing needs

Lesson: Hybrid models and choosing a model

Table: project · what we know · model

Project	What we know	Model
Pacemaker firmware	Life critical, requirement fixed	Waterfall
Spreadsheet product	Modules build on each other	Spiral
First website for a bakery	Customer is unclear about what they want	Prototype
Core banking system	Large and complex, requirement fixed	V model
Food delivery app	Requirements change, customer unclear, modules depend on each other	Agile
Hospital suite for a new client	Modules depend on each other, customer unclear	Hybrid

Lesson: Hybrid models and choosing a model

Table: · scrum · kanban

	Scrum	Kanban
Rhythm	Fixed sprints of 1 to 4 weeks	Continuous flow
Roles	Product owner, Scrum master, developers	No required roles
Limit on work	What fits in the sprint	WIP limit per column
Change during the cycle	Avoided inside a sprint	Allowed any time, if WIP allows
Good for	New product features	Support, maintenance, bug fixing, operations

Lesson: Kanban, scaled Agile and DevOps

Table: model · when testing starts · how often the product is tested · main testing risk

Model	When testing starts	How often the product is tested	Main testing risk
Waterfall	After all coding is finished	Once, in one long phase	Requirement defects found very late and very expensive
V model	Test documents are written from day one, execution after coding	Once per level, in order	Still slow to react to changes
Spiral	In every cycle, module by module	Every cycle, with growing regression	Regression effort grows quickly
Prototype	The prototype itself is reviewed and tested with the customer	Prototype rounds, then normal testing	Customers mistake the prototype for the real thing
Scrum	Day one of each sprint	Every sprint, small pieces	Not enough time for regression without automation
Kanban and DevOps	With every change	Continuously, through the pipeline	Over trust in automation, flaky tests, gaps in exploratory testing

Lesson: Where testing happens in each model

TIPS AND TRAPS

REMEMBER A shop in Uganda can order software from TCS in India. Distance does not matter. What matters is whether the software is built for one customer (service) or for everyone (product).

Lesson: Service based and product based companies

COMMON MISTAKE Even after a careful hand over, problems appear in production. The two usual reasons are an **installation problem** and **customer misuse**. That is why maintenance is a phase of its own.

Lesson: The seven phases, one by one

REMEMBER Waterfall suits **life critical** or **machine critical** applications, where the requirement is fixed and must never be misunderstood: a medical machine, a missile launch system. Apart from waterfall, every other model allows requirement changes and has dedicated testers.

Lesson: The waterfall model

REMEMBER Use the prototype model when the **customer is new to this kind of application** or is **not clear about the requirement**.

Lesson: The prototype model

REMEMBER Choose the V model for **large and complex** applications where quality matters most.

Lesson: The V and V model (V model)

IN THE EXAM Review catches problems before any code exists. Reviewing a CRS that says "login, create mail, read mail, delete mail, view mail" shows that "read mail" and "view mail" mean the same thing, and that registration is missing even though login needs an account.

Lesson: The V and V model (V model)

IN THE EXAM Common interview answers: team size 7 to 8, sprint length 10 to 15 working days, and a one year project (about 250 working days) has about

$$250 \div 15 \approx 16.7$$

so roughly 16 to 18 sprints. One weakness: a small team depends heavily on each member, so losing one person can hurt the project badly.

Lesson: Agile and Scrum

REMEMBER A WIP limit on the Testing column is a quality tool. If testing is full, developers help test or fix bugs, instead of piling more half tested stories in front of the testers.

Lesson: Kanban, scaled Agile and DevOps

1.3 White Box Testing

Before a build ever reaches a tester, the developer checks the code line by line. White box testing looks inside: every path a call can take, every condition, every loop, the memory the code wastes and the time it takes. This page shows how it is done and why it matters to testers too.

KEY IDEAS

- **What white box testing is** White box testing (also called glass box, structural or unit testing) is testing the code itself, line by line, before the program is handed to the testers. It is done by developers, because they can see and understand the code.
- **Unit test programs** A unit test program (UTP) is a small program that calls each function with chosen inputs and checks the result. When every UTP passes, the developer is confident the code is right and hands it over.
- **Path testing** Path testing checks that every path through the code is executed at least once , so that each main function calls the right sub functions in the right order. Developers count the paths with respect to the code .
- **Condition testing** Condition testing checks every condition for both its true and its false outcome. For a login: if the username and password are valid, show the home page, else show an error. One test with valid data and one with invalid data cover both outcomes.
- **Loop testing** Loop testing checks that each loop runs exactly the number of times it should , and that it stops. The notes show three cases: a loop that runs fully, a loop left early with break , and a loop that must not run at all.
- **Statement and branch coverage** Coverage measures how much of the code the tests actually ran:
- **Memory testing** Memory testing checks that the code does not waste memory. The notes give an example: a game for an old Nokia phone had to fit in a tiny amount of memory, so every unused line had to go.
- **Response time testing** Response time testing at code level finds the slow functions . A profiler such as Rational Quantify runs the code and shows which functions take the most time, drawn as thick lines in its call graph.
- **White box versus black box**
- **Stronger coverage: MC/DC and mutation testing** Statement and branch coverage show that code ran. Two stronger ideas check whether the tests would really notice a mistake.

FORMULAS

What white box testing is Exam favourite

$$1 + 2$$

Lesson: What white box testing is

Path testing Exam favourite

$$M = E - N + 2P$$

Lesson: Path testing

Path testing

$$M = \text{number of decisions} + 1$$

Lesson: Path testing

Path testing

$$M = 3$$

Lesson: Path testing

Statement and branch coverage Exam favourite

$$\text{Statement coverage} = \frac{\textit{statements executed}}{\textit{total statements}} \times 100\%$$

Lesson: Statement and branch coverage

Statement and branch coverage

$$\text{Branch coverage} = \frac{\textit{branches taken}}{\textit{total branches}} \times 100\%$$

Lesson: Statement and branch coverage

TABLES

Table: language · common unit testing tool

Language	Common unit testing tool
Java	JUnit, TestNG
.NET (C#)	NUnit, xUnit, MSTest
Python	pytest, unittest
JavaScript	Jest, Mocha

Lesson: Unit test programs

Table: coverage type · every … is executed · for if (a && b)

Coverage type	Every … is executed	For `if (a && b)`
Decision (branch) coverage	branch, both true and false	2 tests: whole condition true, whole condition false
Condition coverage	sub condition, true and false	a true and false, b true and false
MC/DC (used in aviation software)	sub condition shown to change the outcome on its own	3 tests: (T,T), (F,T), (T,F)

Lesson: Condition testing

Table: iterations · why

Iterations	Why
0 (skip the loop)	Empty input must not crash
1 and 2	The first and second passes often hide set up mistakes
A typical value m	The normal case
n − 1, n and n + 1	The edges, where off by one bugs live

Lesson: Loop testing

Table: white box testing · black box testing

White box testing	Black box testing
Done by developers	Done by test engineers
With respect to the code	With respect to the application and the requirement
Code is visible	Code is not visible
Needs programming knowledge	Needs application and domain knowledge
Path, condition, loop, memory and response time testing	Functional, integration, system and acceptance testing
Done first, on each unit	Done after the build is installed on the test server

Lesson: White box versus black box

Table: test · verified · within limit · trusted payee

Test	Verified	Within limit	Trusted payee	Allowed	Shows that
1	Yes	Yes	No	Yes	Baseline
2	No	Yes	No	No	"Verified" alone flips it (compare with 1)
3	Yes	No	No	No	"Within limit" alone flips it (compare with 1)
4	Yes	No	Yes	Yes	"Trusted payee" alone flips it (compare with 3)

Lesson: Stronger coverage: MC/DC and mutation testing

TIPS AND TRAPS

COMMON MISTAKE Checking every function by hand is slow, inaccurate and tedious. A real program has thousands of lines and is changed every day. So developers write code to test their code.

Lesson: What white box testing is

REMEMBER High coverage is a good sign, not a proof. Code can be fully covered by tests that check the wrong expected values.

Lesson: Statement and branch coverage

IN THE EXAM Typical questions: What is white box testing? What are its types? What is a unit test program and who writes it? Why do we need both white box and black box testing? The short answer to the last one: developers prove the code does what they wrote, testers prove it does what the customer asked.

Lesson: White box versus black box

REMEMBER Tools such as PIT (Java), Stryker (JavaScript and C#) and mutmut (Python) do this automatically. A surviving mutant usually points to a missing boundary test, which manual testers know well from boundary value analysis.

Lesson: Stronger coverage: MC/DC and mutation testing

1.4 Functional, Integration and System Testing

A tester first checks each field and button on its own, then the data that flows between modules, then the whole journey from login to logout. Around those three levels sit quick smoke checks, random ad hoc checks and the customer's own acceptance test. This page walks through all of them with the amount transfer and Gmail examples from the notes.

KEY IDEAS

- **Black box testing and the amount transfer example** A black box tester never looks at the code. Everything starts from the requirement, and every object on the screen is tested against it. The notes use an amount transfer page:
- **Functional testing** Functional testing (also called component testing) checks each component of a module, or each module, individually and independently.
- **Integration testing** Integration testing checks the data flow, or interface, between two or more modules. After user A transfers 5,000 to user B, A's balance must drop by 5,000 and B's must grow by 5,000.
- **Types of integration testing, stubs and drivers** When a module is missing during incremental testing, a stand in is used. A stub is a dummy called module (top down: the top module calls a lower module that is not built yet).
- **System testing** System testing checks the end to end flow of the whole application : every module a real user would pass through, from logging in to logging out.
- **Smoke testing** Imagine a new build arrives and the testers start deep testing module by module. On day five they reach compose and find it completely broken: a blocker .
- **Ad hoc and exploratory testing** Real users never read the manual. Ad hoc testing (also called monkey testing) uses the application randomly , the way an unpredictable user would, usually at the end once the planned testing is done. The notes show what it finds on the amount transfer page:
- **User acceptance testing, alpha and beta** UAT is testing done by the customer before accepting the product , to make sure it suits their real business. Buying shoes is the same: you check for damage, try them on, and walk a few steps before paying.
- **The levels of testing at a glance**
- **Positive, negative and end to end testing** Every level of testing mixes two attitudes:
- **Testing below the screen: APIs and system integration** Modern applications are built from many services that talk through APIs (application programming interfaces). The screen is only the last step. A manual tester who can check the API layer finds bugs earlier and pinpoints them faster.

TABLES

Table: field · test value · expected

Field	Test value	Expected
FAN	1234567890 (an existing account)	Accept
FAN	1234567899 (right format, no such account)	Error: account does not exist
FAN	A blocked account, fewer than 10 digits, more than 10 digits, letters, the same number as TAN	Error message for each
Amount	9999, 999, 97, 1	Accept
Amount	10000 (5 digits), −100, "Ten", a space, more than the balance	Error message for each
Transfer	Valid FAN, TAN and amount	"Amount transferred successfully"
Cancel	Any filled form	Every box is cleared

Lesson: Black box testing and the amount transfer example

Table: component · accept · error message

Component	Accept	Error message
To, CC, BCC	Dinga@gmail.com, Dinga_12@gmail.com, upper case, other domains, .co.in, several addresses, copy and paste, blank CC or BCC when To is filled	dingagmail.com (no @), "Ding Dong@gmail.com" (a space)
Subject (up to 200 characters)	200 characters, 1 character, blank, a URL, copy and paste	More than 200 characters, pasted images
Attachment	Browse, different formats and sizes, several files, remove, drag and drop, open the attached file	Files above the size limit
Body (up to 2,000 characters)	2,000 characters, 1 character, blank, a URL, images, formatting, a signature	More than 2,000 characters
Send, Save as draft, Cancel	"Mail sent successfully", "Mail saved in Draft", every field reset	

Lesson: Functional testing

Table: fan entered · expected message

FAN entered	Expected message
1234567890	Accepted
1234567899	FAN A/C# doesn't exist
5555555555	FAN A/C# is blocked
12345	FAN must be 10 digits
12345678901	FAN must be 10 digits
12345abcde	FAN accepts only digits
1234567891 (same as TAN)	FAN and TAN can't be the same
(blank)	FAN can't be blank

Lesson: Functional testing

Table: action in gmail · where the data must arrive

Action in Gmail	Where the data must arrive
Compose and send	Sender's sent items and receiver's inbox
Send to a wrong but valid looking address	A bounce mail in the sender's inbox
Save as draft, or close the compose window	Drafts
Reply or forward from the inbox	Sent items and the receiver's inbox
Move a mail to spam, or delete it	Spam, or trash
Restore from trash	The folder it came from
Chat with an offline user	The user's inbox

Lesson: Integration testing

Table: transfer · message shown · a's balance · b's balance

Transfer	Message shown	A's balance	B's balance	Result
5,000	Amount transferred successfully	20,000 to 15,000	5,000 to 10,000	Pass
10,000	Amount transferred successfully	15,000 to 5,000	10,000 to 10,000	Fail

Lesson: Integration testing

Table: type · how modules are added · use when · example

Type	How modules are added	Use when	Example
Top down (incremental)	From the top module downwards	The data flow is clear and the top drives the rest	IRCTC: registration → login → home → select train → book → payment → logout
Bottom up (incremental)	From the lowest modules upwards	Low level modules are ready first	An HRMS approval moving from employee to team lead to manager to CEO
Sandwich (hybrid)	Top down and bottom up together, meeting in the middle	Large systems with many layers	Both directions in the HRMS
Big bang (non incremental)	All modules at once, in no fixed order	The relationships are not clear or the system is small	Gmail modules tested in any order

Lesson: Types of integration testing, stubs and drivers

Table: approach · order of testing

Approach	Order of testing
Top down	CEO, M1, M2, TL1, TL2, TL3, E1, E2, E3
Bottom up	E3, E2, E1, TL3, TL2, TL1, M2, M1, CEO

Lesson: Types of integration testing, stubs and drivers

Table: formal smoke testing · informal smoke testing

Formal smoke testing	Informal smoke testing
The lead's mail says "start with smoke testing, then FT, IT and ST"	The mail says only "start testing", but a good tester smoke tests anyway

Lesson: Smoke testing

Table: build · smoke result · next step

Build	Smoke result	Next step
B001	Accepted after 7 quick checks	Start functional, integration and system testing
B002	Rejected: Compose fails its basic flow (blocker)	Sent back before any deep testing
B003	Accepted after 7 quick checks	Start functional, integration and system testing

Lesson: Smoke testing

Table: smoke testing · ad hoc testing

Smoke testing	Ad hoc testing
Basic and critical features	Random, unplanned use
As soon as a build is installed	At the last stage
Positive flows only	Mostly negative scenarios
A good way to start	Only if time permits

Lesson: Ad hoc and exploratory testing

Table: who performs uat · when

Who performs UAT	When
Domain expert testers on behalf of the customer	The customer has no testers but pays for experts
One tester from the team with the best product knowledge, at the customer's site	The customer cannot afford domain experts. The tester combines the customer's business knowledge with product knowledge
A few internal employees	The product is used inside the company, for example a courier company's internal tool

Lesson: User acceptance testing, alpha and beta

Table: level · checks · by

Level	Checks	By
Unit (component) testing	Each component or module alone	Developers (white box) and testers (functional testing)
Integration testing	Data flow between modules	Testers
System testing	End to end journeys	Testers
Acceptance testing (UAT)	Business scenarios	The customer

Lesson: The levels of testing at a glance

Table: kind · what it checks · example

Kind	What it checks	Example
Monkey testing	Random input with no plan, to see if the app crashes	Tapping randomly on every screen of a phone app
Gorilla testing	One module tested again and again very hard	Hitting the login page with every kind of input for a whole day
Sanity testing	A quick, narrow check of one fixed area in a new build	After a fix to the OTP, check OTP and login only
End to end testing	A full real journey across all systems	Sign up, add money, pay a bill, see the SMS and the statement

Lesson: Positive, negative and end to end testing

Table: level · what is connected · a typical defect

Level	What is connected	A typical defect
Component integration	Modules inside one application, such as the transfer module and the balance module	Balance not reduced after a transfer
System integration	Our system and outside systems: payment gateway, SMS provider, credit bureau, another bank	Payment succeeds at the gateway but our order still shows "pending"
API testing	One service checked directly, without the screen	The API accepts a negative amount that the screen would block

Lesson: Testing below the screen: APIs and system integration

TIPS AND TRAPS

REMEMBER — The notes also warn: do not **assume** anything the requirement does not say. Is 0 allowed? Is 99.99 allowed? Is 099 the same as 99? If the requirement is silent, ask the BA instead of guessing.

Lesson: Black box testing and the amount transfer example

REMEMBER — Five rules from the notes:

1. While testing one component, keep every other component **valid**, so a failure points at the right field.
2. Start with **positive** values (valid data), then move to **negative** values.
3. Never **over test**: once one value represents a group, testing ten similar values adds nothing.
4. Never **assume** an unclear requirement. Ask.
5. Stop when **maximum test coverage** is reached.

Lesson: Functional testing

IN THE EXAM — The notes add: do not volunteer the types of integration testing in an interview unless you are asked. Explain integration testing itself first.

Lesson: Types of integration testing, stubs and drivers

REMEMBER — System testing is done in an environment as close to production as possible, so that the results reflect what real users will see.

Lesson: System testing

REMEMBER — Is smoke testing mandatory? No, but it is always a good approach. It costs minutes and can save days.

Lesson: Smoke testing

REMEMBER — The Network tab is the easiest start. Open it, do something on the page, and look at the request and response. You will often see that a field the screen hides is still sent, or that an error is swallowed silently. The Manual API Testing subject covers this in depth.

Lesson: Testing below the screen: APIs and system integration

1.5 Applications, Environments and Builds

A tester needs to know where the application lives and how it gets there. This lesson follows a feature from the developer's machine through version control, the build tool and the test server, out to staging and production, and shows where SQL helps a tester skip a 30 day wait.

KEY IDEAS

- **The three parts of an application**
- **Standalone, client server and web applications**
- **Web application architecture** A web application is split across three kinds of servers: web servers hold the pages, application servers run the business logic and database servers hold the tables. Each request only touches the servers it needs.

- **Development, test, staging and production environments** An environment is a set of servers with the software and hardware needed to run the application. Each one has its own purpose and its own URL.
- **SQL for testers, and skipping a 30 day wait** The overdraft is credited 30 days after approval. Testers cannot wait a month. The notes list the options:
- **The build process** A build is the software converted from code into an installable application: a set of features and bug fixes installed on the test server to be tested for stability.
- **Respin, patch and blocker** A blocker stops testing completely. There are two ways to unblock the testers:
- **Release, version and the road to production** A release is the final, stable product handed over to the customer: the whole set of features that was developed, tested and found stable. It also names everything done from requirement to hand over.
- **Test data: realistic, safe and ready** A test is only as good as its data. A transfer test needs accounts with the right balance, the right status and the right history. Test data management is the work of creating, protecting and refreshing that data.
- **Cloud, containers and environments on demand** Not long ago a team shared one QA server for months. Today environments are often created in the cloud in minutes and thrown away after use.

TABLES

Table: part · what it is · example in a bank · visible?

Part	What it is	Example in a bank	Visible?
User interface (UI)	Pages, screens and windows the user sees	The login page, the transfer form	Front end
Business logic (BL)	The code with the rules	"Credit the OD 30 days after approval"	Back end
Database (DB)	Tables that store the data	Accounts, transactions, users	Back end

Lesson: The three parts of an application

Table: · standalone · client server · web

	Standalone	Client server	Web
Where it lives	Fully on the user's machine (.exe)	Client part on the user's machine, the rest on a server	Entirely on servers, opened through a URL
Examples	MS Office, VLC, an antivirus, the OS, a browser	Old chat clients such as GTalk, Yahoo Messenger, desktop Skype	Gmail, Facebook, online banking
Users	One at a time	Many	Many
Installation	On every machine	The client on every machine	None, only a browser
Data sharing	No	Yes	Yes
Main weakness	Maintenance is hard: fixing 1,000 customers abroad means visiting or shipping updates to each	If the server fails, nobody can work	If the server fails, nobody can work

Lesson: Standalone, client server and web applications

Table: step · page shown · servers touched

Step	Page shown	Servers touched
1. Open the URL	Login page	Web
2. Submit username and password	Home page	Web, app, database, app, web
3. Click Compose	Compose page	Web
4. Open the inbox	List of mails	Web, app, database, app, web
5. Click Logout	Login page	Web, app, web

Lesson: Web application architecture

Table: server type · common examples

Server type	Common examples
Web servers	Apache HTTP Server, Nginx, Microsoft IIS, LiteSpeed, Caddy
Application servers	Apache Tomcat, JBoss and WildFly, GlassFish, Jetty, Oracle WebLogic, IBM WebSphere
Database servers	Oracle, Microsoft SQL Server, MySQL, PostgreSQL, IBM DB2

Lesson: Web application architecture

Table: environment · used by · example url · configuration

Environment	Used by	Example URL	Configuration
Development	Developers write code and fix bugs here	`http://devmain1/icici/wipro.local`	Modest (for example 2 GB RAM, one processor)
Test (QA)	Testers test and reproduce bugs here	`http://qamain1/icici/wipro.local`	Similar to development
Staging	System testing on a copy of production	`http://staging/icici/wipro.local`	Same as production
Production (live)	Real users	`www.icici.com`	Highly configured and secured (large RAM and storage, many processors)

Lesson: Development, test, staging and production environments

Table: option · problem

Option	Problem
Change the date on the tester's machine	The server uses its own date, so nothing changes
Change the date on the server	Every other tester is affected
Ask a developer to change 30 days to 2 seconds	The code under test is no longer the real code
Update the data in the database with SQL	Only this test record changes, and the real code runs

Lesson: SQL for testers, and skipping a 30 day wait

Table: record · before the update · after the nightly job runs

Record	Before the update	After the nightly job runs
Dinga	Approved 2026-01-01, not credited	Approved 2025-12-03, credited
Manga	Approved 2026-01-01, not credited	Unchanged, not credited

Lesson: SQL for testers, and skipping a 30 day wait

Table: build · days · new features · bug fixes

Build	Days	New features	Bug fixes	Testing done
B001	1 to 7	5	0	Smoke and functional
B002	8 to 10	3	6	Smoke, functional and integration
B003	11 to 12	2	4	Smoke, functional and integration
B004	13	0	3	Smoke, system testing and retesting
B005	14	0	1	Smoke and retesting, the build is stable

Lesson: The build process

Table: release · version

Release	Version
A major change, often with a new name: Cheeta, Tiger, Lion	A minor change within a release: Cheeta 1.0, 2.0, 3.0

Lesson: Release, version and the road to production

Table: way to get data · how · good · watch out

Way to get data	How	Good	Watch out
Created by hand	Testers enter it through the screens or with SQL	Exactly what the case needs	Slow, and easy to break by other testers
Copied from production	A copy of real data, then masked	Real variety and real volumes	Must be masked before testers see it
Synthetic	Generated by a tool from rules	Safe, any volume, edge cases on demand	May miss oddities of real data
Subsets	A small, consistent slice of a large database	Fast to load and refresh	Links between tables must stay intact

Lesson: Test data: realistic, safe and ready

Table: idea · simple meaning · why testers care

Idea	Simple meaning	Why testers care
Cloud	Servers rented from providers such as AWS, Azure or Google Cloud	Environments can be created and scaled quickly
Container (Docker)	An app packed with everything it needs, so it runs the same everywhere	"Works on my machine" problems shrink
Kubernetes	A system that runs and manages many containers	Explains why a service restarts or moves during a test
Ephemeral environment	A short lived copy of the app for one change or pull request, deleted afterwards	Each story can be tested alone, before it is merged
Infrastructure as code	Environments described in files and built automatically	Test and production are built the same way, so fewer config bugs
Service virtualisation and mocks	A fake version of an outside service, such as a payment gateway	Test even when the real service is down, slow or costs money per call

Lesson: Cloud, containers and environments on demand

TIPS AND TRAPS

REMEMBER — A standalone application is fast and safe from network attacks, but each user's machine has to hold and run everything. Web applications move that load to servers that the company controls and can update for everyone at once.

Lesson: Standalone, client server and web applications

REMEMBER — Why separate environments? Testers need stable data to reproduce bugs, and developers must not change data on the test server. A feature tested on a small machine can behave differently on a large one, so system testing happens on staging, which mirrors production.

Lesson: Development, test, staging and production environments

REMEMBER — Always write the environment and build number in a bug report. With many short lived environments, "it fails on QA" is no longer enough to find the problem again.

Lesson: Cloud, containers and environments on demand

1.6 Static Testing and Reviews

Most bugs are born on paper, long before the first line of code. Static testing finds them there, by reading the requirement, the design or the code with a sharp eye. This page shows how a review works, who does what, and how to spot a requirement that cannot be tested.

KEY IDEAS

- **What static testing is** Static testing means checking a work product without running it. Nobody clicks a button or enters a value. People read the document, the design or the code, and tools scan it, looking for mistakes.
- **What can be reviewed, and what reviews find** Almost anything people write can be reviewed. If a person can read it and understand it, a person can check it.
- **Verification and validation** These two words sound alike and appear in every interview. The classic way to remember them comes from Barry Boehm:
- **Reviewing a requirement like a tester** A tester is often the first person who asks "how would I prove this works?" That question exposes weak requirements. A good requirement has these qualities:
- **The review process in five steps** The ISTQB Foundation syllabus (version 4.0) describes a review as five activities. Small informal reviews squeeze them into a few minutes. A formal inspection follows every step and writes everything down.
- **Who does what in a review** One person can hold several roles in a small team. In a formal inspection each role is a different person. Tap a role in the diagram to read its job.
- **Four types of review, from a chat to an inspection** Reviews range from a quick look by a colleague to a strict, measured inspection. The more formal the review, the more it costs and the more defects it finds per page.
- **Static analysis: tools that read for you** Static analysis is static testing done by a tool. The tool reads the code or a document and reports problems, without running anything. Developers use it most, but testers read its reports and use similar tools on their own documents.
- **Making reviews work in a real team** Reviews fail when they feel like an exam for the author or a waste of time for everyone else. These habits keep them useful:

TABLES

Table: · static testing · dynamic testing

	Static testing	Dynamic testing
Runs the program	No	Yes
What it finds	The defect itself, for example a missing rule in the SRS	A failure, which is then traced back to the defect
When it can start	As soon as anything is written, even day one	Only when a build is ready
Typical methods	Reviews, walkthroughs, inspections, static analysis tools	Functional, integration, system and acceptance testing
Cost to fix what it finds	Very low, the fix is a sentence or a line	Higher, the fix needs code, a new build and retesting

Lesson: What static testing is

Table: where · typical defect found by a review · everyday example

Where	Typical defect found by a review	Everyday example
Requirement	Ambiguity, contradiction, something missing	Page 4 says the PIN has 4 digits, page 9 says 6 digits
Requirement	Not testable	"The app must be fast" has no number
Design	Wrong or missing interface	The design sends the amount in rupees, the payment service expects paise
Code	Unreachable code, unused variables, too complex	An if condition that can never be true
Code	Security weakness	A password written in plain text in a config file
Test case	Wrong expected result, missing negative case	The case expects a 6 letter username to be accepted
Any	Gap in traceability	A requirement with no test case in the RTM

Lesson: What can be reviewed, and what reviews find

Table: · verification · validation

	Verification	Validation
Question	Built right?	Right thing built?
Compared with	The specification, design, standards	The user's real need
Typical activity	Reviews, walkthroughs, inspections, static analysis	Executing the product, UAT, beta, usability sessions
When	All through the project, from the first document	Mostly once something can be used
Done by	Authors, reviewers, testers, QA	Testers, customers, real users

Lesson: Verification and validation

Table: quality · the question a reviewer asks · a bad example

Quality	The question a reviewer asks	A bad example
Clear (unambiguous)	Can it be read only one way?	"Show recent orders" (recent means a day, a week or a month?)
Complete	Is anything missing, such as errors or limits?	Login rules with nothing about a wrong password
Consistent	Does it agree with every other requirement?	Max transfer 50,000 on one page and 1,00,000 on another
Testable	Can I write a pass or fail check for it?	"The screen should look modern"
Feasible	Can it be built with the time and technology?	"Works offline and always shows the live balance"
Traceable	Does it have an id, and can tests point to it?	A rule given only in a phone call
Prioritised	Do we know how important it is?	Fifty features all marked "must have"

Lesson: Reviewing a requirement like a tester

Table: step · what happens · output

Step	What happens	Output
1. Planning	Decide the scope, the goal, the people, the time, and the exit criteria	A short review plan
2. Review initiation	Share the work product, the checklist and the rules	Everyone is ready to start
3. Individual review	Each reviewer reads alone and notes anomalies	Personal issue notes
4. Communication and analysis	Discuss each anomaly, decide if it is a defect, its owner and its severity, and decide if the work product passes	An agreed issue log and a review decision
5. Fixing and reporting	The author fixes the defects, someone checks the fixes, results and metrics are reported	An updated work product and a review report

Lesson: The review process in five steps

Table: role · main job

Role	Main job
Manager	Decides what is reviewed and provides the time and people
Author	Created the work product and fixes the defects found in it
Review leader	Takes overall responsibility, picks the people, organises when and where
Moderator (facilitator)	Runs the meeting so it stays on time, calm and fair to everyone
Scribe (recorder)	Collects the anomalies and writes down decisions and new findings from the meeting
Reviewer	Reads the work product carefully and reports anomalies. Can be a tester, developer, BA, user or expert

Lesson: Who does what in a review

Table: type · led by · how formal · main purpose

Type	Led by	How formal	Main purpose	Example
Informal review	Anyone	No process, no record needed	Find anomalies quickly and cheaply	A developer asks a tester to glance at a test case
Walkthrough	The author	Some preparation, a meeting	Explain, teach, reach a shared understanding, find anomalies	The BA walks the team through the new payments story
Technical review	A moderator	Defined process, records kept	Reach agreement on technical choices, find anomalies	Architects and senior developers review the HLD
Inspection	A trained review leader (never the author)	The most formal, with entry and exit criteria and metrics	Find the most anomalies, measure and improve the process	The SRS of a medical device is inspected line by line

Lesson: Four types of review, from a chat to an inspection

Table: tool type · what it reports · examples

Tool type	What it reports	Examples
Linters and code analysers	Unused variables, unreachable code, risky patterns, broken coding rules	SonarQube, ESLint, Checkstyle, Pylint
Security scanners (SAST)	Passwords in code, injection risks, weak encryption	SonarQube, Semgrep, GitHub CodeQL
Dependency scanners	Libraries with known vulnerabilities or bad licences	Dependabot, Snyk, OWASP Dependency Check
Complexity metrics	Cyclomatic complexity, very long functions, duplicated code	SonarQube, most IDEs
Document checkers	Spelling, grammar, banned vague words, broken links	Spell checkers, link checkers, requirement quality tools
Accessibility checkers	Missing alt text, missing labels, low contrast	axe, Lighthouse, WAVE

Lesson: Static analysis: tools that read for you

TIPS AND TRAPS

REMEMBER Static testing does not replace dynamic testing. It removes cheap defects early so dynamic testing can spend its time on the hard ones.

Lesson: What static testing is

REMEMBER A review finds defects, but it also spreads knowledge. After reading the SRS together, the developer and the tester understand the feature the same way. That alone prevents many bugs.

Lesson: What can be reviewed, and what reviews find

IN THE EXAM In the V model, the left side (reviewing CRS, SRS, HLD and LLD) is mostly verification, and the right side (running the tests, ending with UAT) adds validation.

Lesson: Verification and validation

REMEMBER Most defects are found in step 3, when each person reads alone with full attention. A meeting is for deciding, not for reading the document for the first time.

Lesson: The review process in five steps

COMMON MISTAKE Criticise the work, never the person. "Requirement 4.2 has no maximum length" helps. "You forgot the maximum length again" makes the author defend the document instead of fixing it.

Lesson: Who does what in a review

IN THE EXAM Who leads a walkthrough? The author. Who must never lead an inspection? The author. This one pair of facts answers many exam questions.

Lesson: Four types of review, from a chat to an inspection

REMEMBER Many teams add static analysis to the CI pipeline as a quality gate: if the scan finds a new critical issue, the build is stopped before testers even receive it.

Lesson: Static analysis: tools that read for you

REMEMBER The best compliment for a review: the author says "I am glad you found that now".

Lesson: Making reviews work in a real team

STEP 2 Test design and defects

2.1 Test Case Design and Documents

A tester who keeps everything in their head forgets values, wastes time and tests differently every release. Test documents fix all three. This lesson builds them step by step: scenarios, test cases with real values chosen by proven techniques, review, traceability and the final report.

KEY IDEAS

- **Why test documents, and how they are made** Test documents are written by testers before test execution, while developers are writing the business logic , so that they are ready when the first build arrives.
- **Test scenarios** A test scenario is a high level, one or two line statement of something to test. Each one has a do part and a check part, and scenarios are written module by module. They help a tester understand the flow without worrying yet about exact values.
- **The test case template** A test case is a detailed document with the step by step procedure, the inputs and the expected result for every scenario. Each company has its own template, usually an Excel sheet supplied by the test lead. It has three parts:
- **Error guessing, equivalence partitioning and boundary values** The hardest part of writing test cases is maximum test coverage with a small number of values. Three techniques from the notes:
- **Decision tables, state transitions and pairwise testing** Three more techniques complete a tester's toolbox:
- **Functional, integration and system test cases** The same template holds all three levels. Only the scope changes:
- **Test case review** After every tester finishes writing, test cases are reviewed by another tester, for two reasons: accuracy (maximum coverage) and proper flow , so anyone can reuse them.
- **The repository and the RTM** Approved test cases go into the test case repository , a central place created by the test lead. Once stored they are baselined : written, reviewed and approved, and not changed casually. Smoke test cases are not written separately.
- **The test execution report** The test execution report (TER) is prepared by the lead and handed to the customer. It shows how stable the product is: test cases written, executed, passed and failed per module, with percentages.
- **STLC: the software testing life cycle**
- **Use case testing and checklist based testing** A use case describes how an actor (a user or another system) reaches a goal, step by step. It has a main flow where everything goes well, alternative flows where the user takes another valid path, and exception flows where something goes wrong.
- **Writing test cases that others can run** A test case is written once and run many times, often by someone else. These habits make it useful for years.

FORMULAS

Error guessing, equivalence partitioning and boundary values

Exam favourite

$$a - 1,\ a,\ a + 1,\ b - 1,\ b,\ b + 1$$

Lesson: Error guessing, equivalence partitioning and boundary values

TABLES

Table: # · do · check

#	Do	Check
1	Enter a valid username and password and click OK	The home page is displayed
2	Enter an invalid username or password	An error message is displayed
3	Leave both fields blank and click OK	An error message is displayed
4	Enter values and click Cancel	Both fields are reset
5	Log in with valid details	The username appears on the home page
6	Compose a mail and send it	A confirmation message is displayed
7	Compose and send a mail	It is in the receiver's inbox and the sender's sent items
8	Compose a mail and save it as a draft	A confirmation message is displayed and the mail is in drafts

Lesson: Test scenarios

Table: part · fields

Part	Fields
Header	Test case name, type (FT, IT, ST), requirement number, module, status, severity, pre condition, test data, release, version, brief description
Body	Step number, description, inputs, expected result, actual result, status (pass if expected equals actual), comments
Footer	Author, date, reviewed by, approved by

Lesson: The test case template

Table: build · step results · status

Build	Step results	Status
B001	Pass, pass, pass, pass	Pass
B002	Pass, pass, pass, fail (a minor step)	Partial
B003	Pass, pass, fail (a main step), pass	Fail

Lesson: The test case template

Table: technique · idea · for an amount from 5,000 to 50,000

Technique	Idea	For an amount from 5,000 to 50,000
Error guessing (EG)	Guess values from experience: blank, decimals, letters, copy and paste	6,000 ✓, 51,000 ×, blank ×, 12.50 ×, "Ten" ×
Equivalence partitioning (EP)	Split inputs into classes that behave the same and test one value from each	One valid (6,000) and two invalid (4,000 and 61,000)
Boundary value analysis (BVA)	Bugs hide at the edges, so test each boundary and its neighbours	4,999 ×, 5,000 ✓, 5,001 ✓, 49,999 ✓, 50,000 ✓, 50,001 ×

Lesson: Error guessing, equivalence partitioning and boundary values

Table: input type · ep rule · example

Input type	EP rule	Example
Range	One valid, two invalid (below and above)	Amount 5,000 to 60,000: 6,000 ✓, 4,000 ×, 61,000 ×
Set	One valid, two invalid	Product id in {1 … 5}: 1 ✓, 0 ×, 6 ×
Boolean	True and false	A true or false question: select each option once

Lesson: Error guessing, equivalence partitioning and boundary values

Table: band · values to test · expected result

Band	Values to test	Expected result
Below the range	99,999	Loan amount should be 1 to 7 lakh
1 to 2 lakh	1,00,000, 1,00,001, 1,99,999, 2,00,000	5.68% interest
Above 2 to 5 lakh	2,00,001, 2,00,002, 4,99,999, 5,00,000	3.29% interest
Above 5 to 7 lakh	5,00,001, 5,00,002, 6,99,999, 7,00,000	1.99% interest
Above the range	7,00,001	Loan amount should be 1 to 7 lakh

Lesson: Error guessing, equivalence partitioning and boundary values

Table: technique · use it when · example

Technique	Use it when	Example
Decision table	The outcome depends on a combination of conditions	Valid user? Valid password? Account locked? Each combination is one rule with one expected action
State transition	The system moves between states and only some moves are allowed	An account: active, then locked after 3 wrong passwords, then active again after reset. A bug life cycle is another example
Pairwise (all pairs)	Many parameters with many values make every combination impossible	OS × browser × language: cover every pair of values instead of every triple

Lesson: Decision tables, state transitions and pairwise testing

Table: valid user · valid password · account locked · expected result

Valid user	Valid password	Account locked	Expected result
Y	-	Y	Account locked
Y	Y	N	Home page
-	N	N	Invalid username or password
N	Y	-	Invalid username or password
N	N	Y	Invalid username or password

Lesson: Decision tables, state transitions and pairwise testing

Table: · ft case: amount transfer · it case: amount transfer · st case: overdraft

	FT case: amount transfer	IT case: amount transfer	ST case: overdraft
Name	`Cheeta_2.0_ICICI_AT_FT`	`Cheeta_2.0_ICICI_AT_Int`	`Bhalu_3.0_ICICI_OD_ST`
Pre condition	1 valid login, 2 valid accounts, minimum balance	2 valid logins, 2 valid accounts	2 valid logins (user and manager), employee id, salary, database access
Focus	Every value of FAN, TAN and amount, Transfer and Cancel	A's balance falls and B's rises by the amount	Apply, approve, credit after 30 days, repay with interest and fee
Typical steps	8	8	22

Lesson: Functional, integration and system test cases

Table: step · severity · review comment · author's response

Step	Severity	Review comment	Author's response
Header	Critical	Pre condition missing	Not fixed, with a reason: none is needed
5	Critical	Positive values missing for FAN	Fixed
9	Major	Need more positive values	Fixed
14	Minor	Need more negative values	Fixed
22	Minor	Sentence not clear	Fixed

Lesson: Test case review

Table: when · who

When	Who
After test cases are reviewed and stored, before execution starts. Not while writing (incomplete) and not during review (cases may be rejected)	Every tester for their own modules. The lead consolidates them for the release

Lesson: The repository and the RTM

Table: requirement · test cases

Requirement	Test cases
R1	TC01, TC02
R2	TC03
R3	No test case
R4	TC04, TC05, TC06
R5	TC07
R6	TC08
R7	TC09, TC10
R8	TC11
R9	TC12
R10	TC13, TC14

Lesson: The repository and the RTM

Table: module · written · executed · pass

Module	Written	Executed	Pass	Fail	Pass %	Fail %
Loans	480	480	476	4	99.2%	0.8%
Sales	490	490	490	0	100.0%	0.0%
Market	370	370	368	2	99.5%	0.5%
Total	**1340**	**1340**	**1334**	**6**	**99.6%**	**0.4%**

Lesson: The test execution report

Table: phase · what happens

Phase	What happens
System study	Read, review and understand the requirement
Test plan	Leads and managers plan the whole testing activity
Test documents	Write scenarios and cases, fix review comments, get approval, store in the repository
RTM	Trace every requirement to test cases
Test execution	Run the cases on each build
Defect tracking	Log, follow and retest bugs
Test execution report	Report stability to the customer
Retrospect (post mortem)	What went well, what went wrong, what to improve

Lesson: STLC: the software testing life cycle

Table: flow · steps of "withdraw cash at an atm" · test idea

Flow	Steps of "Withdraw cash at an ATM"	Test idea
Main	Insert card, enter PIN, choose amount, take cash, take card	Withdraw ₹2,000 with enough balance
Alternative	Choose "other amount" instead of a quick amount	Withdraw ₹1,500 using other amount
Alternative	Ask for a receipt	Receipt shows the right amount and balance
Exception	Wrong PIN three times	Card is blocked with a clear message
Exception	Amount above the balance	"Insufficient balance", no cash, no debit
Exception	Cash not taken within 30 seconds	Cash is taken back and the account is not debited

Lesson: Use case testing and checklist based testing

Table: mistake · example · better

Mistake	Example	Better
Vague step	Enter details and submit	Enter amount 5,000 and account 1234567890, then click Transfer
Vague expected result	It should work	Message "Transfer successful" and balance reduced by 5,000
Many checks in one case	Test login, transfer and logout	One goal per case, so a failure points to one thing
Hidden dependency	Uses the account created in case 12	Preconditions list the data the case needs
Only happy paths	Valid amounts only	Add boundaries, invalid classes and error recovery
Copied UI text that changes	Click the blue button	Click Transfer (the button name, not its look)

Lesson: Writing test cases that others can run

TIPS AND TRAPS

REMEMBER — Product based companies usually write scenarios first. Service based companies often go straight to test cases.

Lesson: Test scenarios

REMEMBER — Rules of thumb from the notes: for maximum coverage always add error guessing to any technique. Use BVA plus EG for ranges, EP plus EG for sets and booleans, and plain EG for simple fields such as "exactly 3 digits".

Lesson: Error guessing, equivalence partitioning and boundary values

IN THE EXAM Interview tip from the notes: if asked to write test cases for an unknown application (ERP, insurance), ask for the requirement first. For a known one (Gmail), write scenarios. For an object (a pen, a chair, an ATM), think of functional, integration, system, compatibility, performance and usability checks.

Lesson: Functional, integration and system test cases

REMEMBER Review ethics: aim for maximum coverage, point out mistakes rather than writing the solutions, review the content and not the author, and remember that both are responsible. If a scenario was never written and the reviewer missed it, both share the blame. If it was written but skipped while testing, the author is responsible.

Lesson: Test case review

COMMON MISTAKE **Drawback of the RTM:** it only ensures that every requirement has at least one test case. It does not guarantee that all the test cases a requirement needs are there.

Lesson: The repository and the RTM

REMEMBER If the test plan's exit criterion is 97% and the TER shows 99.6%, the release can go ahead.

Lesson: The test execution report

IN THE EXAM Typical interview numbers from the notes: about 3 to 4 test cases written a day and 2 to 3 executed, depending on complexity, and roughly 90 to 120 test cases for a two to three month project.

Lesson: STLC: the software testing life cycle

REMEMBER Read your own case as if you joined the team today. If you would need to ask a question to run it, the answer belongs in the case.

Lesson: Writing test cases that others can run

2.2 Defect Management

Finding a bug is half the job. The other half is reporting it so clearly that a developer can reproduce it at once, and following it until it is closed. This lesson walks through every status a bug can reach and why, how to rate severity and priority, and how to write a bug report that cannot be misread.

KEY IDEAS

- **Defect, bug, error, issue and failure** A chain links them: a developer makes an error , the code now contains a defect , and when that code runs it can cause a failure . Defects come from three kinds of implementation:
- **The defect life cycle** Every bug moves through a fixed set of statuses. Only certain moves are allowed, and each move is made by a particular person.
- **Who gets the bug**
- **Invalid and duplicate** The developer does not accept the bug because it is not against the requirement. There are two outcomes:
- **Not reproducible** The developer accepts that it might be a bug but cannot make it happen. The notes list six reasons. Pick one in the widget to compare the tester's setup with the developer's.
- **Can't fix, deferred and RFE**
- **Severity and priority** The developer works through the queue by priority first and severity second:
- **Writing the bug report** A good bug report lets a developer reproduce the bug without asking a single question. The template fields:
- **Bug tracking tools and queries** The tool generates the bug id, fills in the reporter and date, and offers drop downs for module, status, severity and priority.
- **The defect triage meeting** When many bugs arrive at once, not all can be fixed at once. Triage , a word borrowed from hospital emergency rooms, means deciding quickly which bugs need attention first. Most teams hold a short triage meeting daily or a few times a week during a test cycle.

- **Root cause analysis: 5 whys and the fishbone** Fixing a bug removes one defect. Finding its root cause stops the same kind of bug from coming back. Two simple techniques are used everywhere, from car factories to software teams.

TABLES

Table: word · meaning · who usually says it

Word	Meaning	Who usually says it
Defect	A deviation from the requirement	Formal word in reports
Bug	The informal name for a defect	Testers
Error	A mistake in the code	Developers
Issue	A problem the customer faces	Customers
Failure	What the user sees when defects make the software stop working as needed	Everyone

Lesson: Defect, bug, error, issue and failure

Table: kind · example with a login page that needs username, password, ok and cancel

Kind	Example with a login page that needs username, password, OK and Cancel
Wrong implementation	Cancel submits the form instead of clearing it
Missing implementation	There is no Cancel button at all
Extra implementation	A "Forgot password" link appears that was never asked for

Lesson: Defect, bug, error, issue and failure

Table: status · set by · meaning

Status	Set by	Meaning
New or Open	Tester	The bug has just been reported
Assigned	Tester (or lead)	A developer now owns it
Fixed	Developer	The code was changed, and the fix will come in a later build
Closed	Tester	Retesting on the new build shows the bug is really fixed
Reopen	Tester	Retesting shows the bug is still there, so it goes back to the developer

Lesson: The defect life cycle

Table: bug · statuses in order · final status

Bug	Statuses in order	Final status
Bug#1	New, Assigned (tester), Fixed (developer), Closed (tester)	Closed
Bug#2	New, Assigned, Fixed, Reopen (tester), Assigned, Fixed, Closed (tester)	Closed

Lesson: The defect life cycle

Table: assign to · when

Assign to	When
The developer	You know who wrote the module (M1 is D1's, M2 is D2's)
The development lead	You do not know which developer owns it. The lead forwards it
The test lead	The test team has no direct contact with developers, or a lead reviews bugs from new testers first. Safer, but slower

Lesson: Who gets the bug

Table: new problem found · closest match in the repository · what the tester does

New problem found	Closest match in the repository	What the tester does
Send button does nothing after attaching a pdf file	Bug#9, very similar, status Assigned	Do not log it. Add a comment to Bug#9
Draft loses the mail body when saving	Bug#17, similar, status Closed	Log a new bug and mention Bug#17
Search returns no results	No match	Log it and CC all testers

Lesson: Invalid and duplicate

Table: cause · what went wrong · put this in the report

Cause	What went wrong	Put this in the report
Incomplete steps	The tester forgot to mention "save as draft first"	Every step, in order
Server mismatch	Found on the test server, tried on a development server	The server URL
Platform mismatch	Found on XP with Internet Explorer, tried on Windows 7 with Firefox	OS and browser with versions
Data mismatch	Found as a normal user, tried as an admin who has more rights	The test data and login used
Build mismatch	Found in B001, tried in B002 where another fix changed the code	The build number
Inconsistent bug	Appears only sometimes	A screenshot or recording the first time it appears

Lesson: Not reproducible

Table: status · accepted? · reproduced? · why

Status	Accepted?	Reproduced?	Why
Invalid	No	Not needed	Not against the requirement
Duplicate	Yes	Already known	Another tester logged it first
Not reproducible	Yes	No	Missing details or a different setup
Can't fix	Yes	Yes	A minor bug in the core of the code, fixing costs more than living with it, or no technology support
Deferred	Yes	Yes	A minor bug postponed to the next release because time is short. Never in the early builds
RFE	A suggestion	Not a bug	A request for enhancement, such as adding a "Register" link the requirement does not mention

Lesson: Can't fix, deferred and RFE

Table: order · priority · severity · bug

Order	Priority	Severity	Bug
1	P1	Blocker	Transfer shows a blank page
2	P1	Critical	Confirmation shown but balance is wrong
3	P2	Major	Sent mail missing from Sent items
4	P2	Minor	Typo in the company slogan on the home page
5	P3	Major	No confirmation, balance correct
6	P4	Minor	Field labels misaligned

Lesson: Severity and priority

Table: report · ready to send? · what is missing

Report	Ready to send?	What is missing
Bug#31, Sent items on B004, Windows 7, Chrome 30	Yes	Nothing. Severity, priority, build, platform, steps, observation and expected result are all there
Bug#32	No	Priority, build, server, platform, the expected result, and more than one step to reproduce

Lesson: Writing the bug report

Table: search · result

Search	Result
Fixed, waiting for retest on the new build	Bug#31, Bug#52, Bug#74
Reopened	Bug#40 (Compose)
Still open, by severity	5 of 8 bugs: 1 blocker, 2 critical, 1 major, 1 minor

Lesson: Bug tracking tools and queries

Table: term · meaning

Term	Meaning
Defect clustering	A small number of modules contain most of the defects
Hot fix	An immediate fix (within hours) of a high priority bug, for example one found by the client the day before release
Latent bug	A bug that was present but not found in earlier versions
Golden bug	A bug that appears every time, with high severity and high priority

Lesson: Bug tracking tools and queries

Table: who attends · brings

Who attends	Brings
Test lead or tester	The new bugs, how to reproduce them, the impact on testing
Development lead	How hard each fix is and who can take it
Product owner or business analyst	How much each bug matters to users and the business
Release or project manager	Dates, and what can still fit in this release

Lesson: The defect triage meeting

Table: decision · when

Decision	When
Fix now	Blockers and critical bugs, anything that stops testing or the release
Fix in this release	Major bugs with real user impact
Defer to a later release	Low impact, risky to fix late, a workaround exists
Need more information	Cannot reproduce, unclear expected result
Reject	Works as designed, duplicate, not a defect

Lesson: The defect triage meeting

Table: root cause found · fix for this bug · fix that prevents the next one

Root cause found	Fix for this bug	Fix that prevents the next one
Requirement said "above 5,000" without saying if 5,000 is included	Correct the comparison	Add boundary wording to the requirement review checklist
No test for exactly 5,000	Add the test	Make boundary value analysis part of the Definition of Done
Test data had no account at the limit	Create one	Keep standard edge accounts in every environment

Lesson: Root cause analysis: 5 whys and the fishbone

TIPS AND TRAPS

REMEMBER Extra features are defects too. Nobody tested them against a requirement, nobody planned their security, and they can break other features.

Lesson: Defect, bug, error, issue and failure

REMEMBER A bug found while retesting a different bug is a **new** bug with its own number. Reopen is only for the same bug coming back.

Lesson: The defect life cycle

REMEMBER If a similar bug exists and is still open, assigned, fixed or reopened, do not log it again. Add a comment instead. If the old bug is closed and the problem is back, log a new bug and mention the old number.

Lesson: Invalid and duplicate

IN THE EXAM Classic interview pair: **high severity, low priority** (a crash in a report used once a year) and **low severity, high priority** (the company name misspelled on the home page). Severity and priority are related but not the same.

Lesson: Severity and priority

COMMON MISTAKE Keep the description factual: what you did, what you saw, what you expected. Never write "the developer broke the inbox again".

Lesson: Writing the bug report

REMEMBER Come to triage prepared: one line per bug that says what breaks for the user. "Customers cannot pay with UPI on Android 14" decides faster than "Exception in PaymentActivity".

Lesson: The defect triage meeting

REMEMBER Root cause analysis looks for causes in the process, never for a person to blame. "Who did it?" stops learning. "What let it happen?" starts it.

Lesson: Root cause analysis: 5 whys and the fishbone

STEP 3 Regression and non functional testing

3.1 Regression, Retesting and Automation

Every change to the code can break something that used to work. Regression testing re-runs old test cases to prove it did not. This lesson shows how to pick which old cases to run, why that work moves to automation from the second release on, and how to tell the many look alike terms apart.

KEY IDEAS

- **What regression testing is** Each new release adds features, removes some, or modifies others. All three mean the code changes, and any code change can break something that already worked.

- **Unit, regional and full regression** Follow the dependencies one step at a time to find the impact area of a change:
- **Impact analysis and the regression suite** No single person sees every impact, so three groups each send a report:
- **Retesting versus regression** Regression happens at two levels. Across releases because of new or changed features, and across builds because of bug fixes: B001 has bugs, the developer changes the code to fix them, and on B002 the tester checks the fixes and the areas around them.
- **Why regression moves to automation** Regression is the most repetitive work in testing, which makes it the best candidate for automation. A tool runs the same steps in seconds, does not get bored, and can run every night.
- **The automation life cycle** A night of script runs on a new build, and what the team does with each failure:
- **What cannot be automated**
- **The classic comparisons**
- **Choosing what to automate first** Nobody automates everything at once. A simple score helps a team pick the cases that pay back fastest:
- **Flaky tests and why they matter** A flaky test passes and fails on the same code without any change. Run it ten times and you might see eight green and two red. Flaky tests are dangerous because people learn to ignore red results, and then a real failure is ignored too.

TABLES

Table: release · new test cases · regression cases · total run

Release	New test cases	Regression cases	Total run	Repository	Regression share
1	300	0	300	300	0%
2	250	150	400	550	37%
3	320	300	620	870	48%

Lesson: What regression testing is

Table: challenge · why

Challenge	Why
Finding the impact area	A change in one module can affect modules that look unrelated
Time	The number of test cases grows every release, while the team often does not
Accuracy	Running the same cases again and again is monotonous, and attention drops

Lesson: What regression testing is

Table: type · what is tested · when

Type	What is tested	When
Unit regression	Only the changed feature	A small, isolated change. Login changes from 3 letters to 3 digits, so only login cases are run
Regional regression	The changes plus the **impact area** found by impact analysis	The usual case. A change in Compose is checked together with Inbox and Sent items
Full regression	The changes plus every old feature	Most of the product changed, or the application is life or machine critical. A release after three years with 70% of the code rewritten

Lesson: Unit, regional and full regression

Table: changed module · impact area

Changed module	Impact area
Compose	Sent items, Drafts and Inbox (step 1), Search (step 2)
Settings	Inbox (step 1), Search (step 2)
Login	Home (step 1), Compose, Inbox and Chat (step 2), Sent items, Drafts and Search (step 3)

Lesson: Unit, regional and full regression

Table: who · knowledge they bring

Who	Knowledge they bring
Customer	Business knowledge: which business flows use the changed feature
Developers	Coding knowledge: which code calls the changed code
Testers	Product knowledge: which screens and flows the user sees together

Lesson: Impact analysis and the regression suite

Table: feature · flagged by · test cases picked

Feature	Flagged by	Test cases picked
F1	Customer, developer	TC#1 to TC#12 (12)
F2	Customer	TC#13 to TC#20 (8)
F3	Developer	TC#21 to TC#29 (9)
F7	Customer	TC#61 to TC#69 (9)
F8	Developer	TC#70 to TC#77 (8)
F9	Tester	TC#78 to TC#89 (12)
F27	Tester	TC#240 to TC#250 (11)
F29	Tester	TC#260 to TC#265 (6)

Lesson: Impact analysis and the regression suite

Table: tool type · examples · licence

Tool type	Examples	Licence
Functional testing	Selenium, UFT (formerly QTP), WinRunner, SilkTest, Playwright, Cypress	Selenium, Playwright and Cypress are open source, UFT is licensed
Performance testing	LoadRunner, JMeter, NeoLoad, WebLOAD, Gatling, k6	JMeter, Gatling and k6 are open source, LoadRunner is licensed

Lesson: Why regression moves to automation

Table: runs · manual (minutes) · automated (minutes)

Runs	Manual (minutes)	Automated (minutes)
1	60	121
2	120	122
3	180	123, now cheaper
50	3,000 (50 hours)	170 (2.8 hours)

Lesson: Why regression moves to automation

Table: test case · script · status · next step

Test case	Script	Status	Next step
TC#1	TS#1	Pass	
TC#2	TS#2	Pass	
TC#3	TS#3	Fail	Checked by hand: a real bug. Report it and rerun the script after the fix
TC#4	TS#4	Pass	
TC#5	TS#5	Fail	Checked by hand: the application is fine. Update the script
TC#150	TS#150	Fail	Checked by hand: a real bug. Report it and rerun the script after the fix

Lesson: The automation life cycle

Table: reason · examples

Reason	Examples
The feature is unstable	New features, or features that are still changing. A script written today breaks tomorrow
No technology support	Captcha, a virtual keyboard, a one time password sent to a phone
Too complex or too human	Flows with many manual decisions, usability and look and feel, ad hoc testing

Lesson: What cannot be automated

Table: smoke testing · sanity testing

Smoke testing	Sanity testing
As soon as a build is installed	When bug fixes arrive
Shallow and wide: every main feature at a high level	Narrow and deep: one area in detail
Can be documented and scripted	Usually not documented

Lesson: The classic comparisons

Table: static testing · dynamic testing

Static testing	Dynamic testing
Without executing the code: reviews, walk throughs, inspections	By executing the code: unit, integration, system and acceptance testing
Verification	Validation

Lesson: The classic comparisons

Table: quality assurance (qa) · quality control (qc)

Quality assurance (QA)	Quality control (QC)
Process oriented	Product oriented
Are we developing it right?	Did we develop it right?
Reviews and walk throughs (verification)	Unit, integration, system and acceptance testing (validation)
Preventive: stops bugs before the product is built	Detective: finds bugs after it is built

Lesson: The classic comparisons

Table: functional testing · non-functional testing

Functional testing	Non-functional testing
Does each feature work as the requirement says?	How well does it work: speed, ease of use, platforms
Unit, integration, system, acceptance	Performance, usability, compatibility, security
Expected results can be written in advance	Results are measured, then compared with targets

Lesson: The classic comparisons

Table: factor · score high when · example

Factor	Score high when	Example
Frequency	The case runs every build or every release	Login, smoke checks, main payment journey
Business risk	A failure would hurt many users or cost money	Fund transfer, checkout, interest calculation
Stability	The feature and its screens rarely change	An address form unchanged for two years
Effort to run by hand	Many data sets or many browsers	The same rule checked for 40 tariff plans
Effort to automate (subtract)	It is easy to automate and to maintain	An API check is easier than a drag and drop screen

Lesson: Choosing what to automate first

Table: common cause · example · typical fix

Common cause	Example	Typical fix
Timing	The test clicks before the page finished loading	Wait for a condition, never a fixed sleep
Shared data	Two tests use the same account at the same time	Each test creates or owns its data
Test order	A test only passes if another test ran first	Make tests independent
Environment	A slow or overloaded test server, an outside service that times out	Stable environments, mocks for outside services
Real product bug	A race condition in the app itself	Report it as a bug. Users will meet it too

Lesson: Flaky tests and why they matter

TIPS AND TRAPS

REMEMBER **Regression testing** is re-executing old test cases across releases (or builds) to make sure that changes have not affected existing functionality. The old test cases that are picked again are **regression test cases**. They are taken from the repository, not written again.

Lesson: What regression testing is

IN THE EXAM If an interviewer asks which regression you do, the usual answer is **regional regression**, with the impact area found by impact analysis.

Lesson: Unit, regional and full regression

REMEMBER Retesting answers "is this bug gone?". Regression answers "did the fix break anything else?". Both are done on every new build.

Lesson: Retesting versus regression

REMEMBER Automate when there are multiple releases, many regression cycles or repetitive tasks. That normally starts from the **second release**, once there are stable old features to protect.

Lesson: Why regression moves to automation

COMMON MISTAKE Automation does not replace testers. Someone still writes the test cases, reviews failures, and does the exploratory and usability testing that needs human judgement.

Lesson: What cannot be automated

REMEMBER As a manual tester, if an automated check is flaky, try the same steps by hand quickly and repeatedly. If you can make it fail, the flakiness may be a real bug in the product, not in the test.

Lesson: Flaky tests and why they matter

3.2 Non-functional Testing

An application can do everything the requirement asks and still fail its users: it breaks on their browser, confuses them, or slows to a crawl when a thousand people log in at once. Non-functional testing measures how well the product works, once it already works.

KEY IDEAS

- **What non-functional testing covers** Functional testing asks whether each feature does what the requirement says. Non-functional testing asks how well the product does it: on which platforms, how easily, and how fast under real load.
- **Compatibility testing** Compatibility testing checks the application on different software and hardware platforms, because real users do not all have the same machine.
- **Virtual machines and the compatibility process** The notes compare three ways to get many platforms. Giving each tester one OS means nobody can check another. Installing several OS on each machine means restarting all day.
- **Usability testing** Usability is user friendliness: important features are easy to reach, the language is easy to understand, the look and feel is consistent, navigation takes few steps, and the effort to learn is small.
- **Performance testing** Performance testing checks how the application behaves when load is applied. Three factors matter:
- **Bottlenecks and tuning** If the average response time is 6.3 seconds against a target of 5, there is a bottleneck . Performance bugs are not reproduced like functional bugs. They are analysed and tuned:
- **Load, stress, scalability, volume and soak** At 1,000 users the system is now 2.5 seconds slower than before the stress. It did not recover, so the team looks for a memory leak.
- **The quality model: ISO/IEC 25010:2023** "Non functional" is a big bag. The international standard ISO/IEC 25010 sorts product quality into named characteristics, and its 2023 edition updated the list. Teams use it as a checklist so that no kind of quality is forgotten in the test plan.
- **Reliability, recovery and installation testing** Users forgive a slow day. They do not forgive lost money or lost data. These tests check that the product keeps working, and comes back safely when something breaks.

FORMULAS

Performance testing Exam favourite

$$1000 \times 30 \times 24 = 720{,}000$$

Lesson: Performance testing

Load, stress, scalability, volume and soak

Exam favourite

$$\text{users} = \text{throughput} \times (\text{response time} + \text{think time})$$

Lesson: Load, stress, scalability, volume and soak

Load, stress, scalability, volume and soak

$$\frac{1000}{20} = 50$$

Lesson: Load, stress, scalability, volume and soak

Reliability, recovery and installation testing

Exam favourite

$$\text{availability} = \frac{MTBF}{MTBF + MTTR}$$

Lesson: Reliability, recovery and installation testing

TABLES

Table: platform · share of users · users covered so far

Platform	Share of users	Users covered so far
Windows + Chrome (base)	41%	41%
Android + Chrome	16%	57%
Windows + Edge	14%	71%
iOS + Safari	11%	82%
macOS + Safari	5%	87%
macOS + Chrome	4%	91%

Lesson: Compatibility testing

Table: forward compatibility · backward compatibility

Forward compatibility	Backward compatibility
Works on newer versions: Windows 10 to Windows 11, Chrome 120 to Chrome 130	Works on older versions: Windows 11 to Windows 10, Android 14 to Android 11

Lesson: Virtual machines and the compatibility process

Table: page · checks passed · failed check to report

Page	Checks passed	Failed check to report
Login	3 of 3 (two checks do not apply)	None
Home	3 of 4	Alt text on images
Sales	4 of 5	Consistent fonts
Market	4 of 5	Breadcrumb

Lesson: Usability testing

Table: factor · meaning · example target

Factor	Meaning	Example target
Response time	Time the server takes to respond to a request	Under 5 seconds
Load	Number of users at the same time	1,000 users
Stability	Users working continuously over a period	24 hours

Lesson: Performance testing

Table: scenario · share · virtual users

Scenario	Share	Virtual users
S1: login, compose, logout	40%	400
S2: login, inbox, download, logout	35%	350
S3: login, chat, logout	25%	250

Lesson: Performance testing

Table: measure · value

Measure	Value
Fastest	3.8 s
Slowest	7.0 s
Average	4.70 s
50th percentile	4.6 s
90th percentile	5.2 s
95th percentile	5.2 s

Lesson: Performance testing

Table: where a bottleneck can be · typical tuning

Where a bottleneck can be	Typical tuning
Code	A slow loop or too many database calls
Database	A missing index, a heavy query
Server software	Thread pool or connection pool limits on the web or application server
Hardware	Too little RAM or CPU
Network	Low bandwidth, too many round trips

Lesson: Bottlenecks and tuning

Table: type · what is applied

Type	What is applied
Load testing	Load up to the desired level (1,000 users with a 5 second goal)
Stress testing	Load beyond the desired level, to see how and where it breaks
Scalability testing	Load increased or decreased step by step to find the limit
Stability testing	The desired load held for a long time
Volume testing	A huge volume of data, such as millions of rows
Soak (endurance) testing	Load for a very long time on a minimal set up, to find leaks that appear only after hours

Lesson: Load, stress, scalability, volume and soak

Table: direction · users and response time · what it tells you

Direction	Users and response time	What it tells you
Upward	1,000: 4.5 s, 1,100: 6.0 s, 1,200: 6.5 s, 1,300: 8.0 s, 1,400: 8.5 s, 1,500: crash	Meets 5 s up to 1,000 users and crashes at 1,500
Downward	1,000: 7.0 s, 900: 6.8 s, 800: 5.6 s, 700: 4.9 s	After the crash it meets 5 s only up to 700 users

Lesson: Load, stress, scalability, volume and soak

Table: characteristic · question it asks · example test

Characteristic	Question it asks	Example test
Functional suitability	Does it do the right things, correctly and completely?	All functional testing
Performance efficiency	Is it fast enough with the expected load and resources?	Load, stress and soak tests
Compatibility	Does it work with other systems and on other platforms?	Browser and device testing, integrations
Interaction capability (was usability)	Can people learn and use it, including people with disabilities?	Usability sessions, accessibility testing
Reliability	Does it keep working, and recover from faults?	Soak, failover and recovery tests
Security	Is data protected from people who should not access it?	Access control, injection, session tests
Maintainability	Can it be changed easily and safely?	Code reviews, static analysis
Flexibility (was portability)	Can it be moved, installed and scaled to new environments?	Installation, upgrade and scalability tests
Safety (new in 2023)	Does it avoid putting people or property in danger?	Hazard analysis, fail safe tests in medical or car software

Lesson: The quality model: ISO/IEC 25010:2023

Table: test · what it checks · example

Test	What it checks	Example
Reliability	The system works without failure for a long time under normal use	Run normal traffic for 72 hours and count failures
Recovery	After a crash, power cut or network loss, the system comes back with correct data	Pull the network cable during a transfer, then check both balances
Failover	When one server dies, another takes over without users noticing	Switch off the main database node during a load test
Backup and restore	Backups can really be restored, within the promised time	Restore last night's backup to a test server and compare records
Installation	Fresh install, upgrade, repair and uninstall all work	Upgrade from version 3.2 to 4.0 and check that settings and data survive
Disaster recovery	The service can move to another data centre or region	A planned drill moving all traffic to the backup region

Lesson: Reliability, recovery and installation testing

Table: availability · allowed downtime per year · per month (30 days)

Availability	Allowed downtime per year	Per month (30 days)
99%	about 3.65 days	about 7.2 hours
99.9%	about 8.76 hours	about 43 minutes
99.99%	about 53 minutes	about 4.3 minutes
99.999%	about 5.3 minutes	about 26 seconds

Lesson: Reliability, recovery and installation testing

TIPS AND TRAPS

REMEMBER

Non-functional testing starts once the product is **functionally stable**. Measuring the speed of a feature that still gives wrong answers wastes everyone's time.

Lesson: What non-functional testing covers

REMEMBER Not every application needs it. An internal tool where all 1,000 employees use the same company laptop and browser can skip compatibility testing. A public site like a social network cannot.

Lesson: Virtual machines and the compatibility process

REMEMBER The average hides slow requests. Teams also report the **90th percentile**: the time within which 90% of requests finished.

Lesson: Performance testing

IN THE EXAM Little's law links the numbers:

$$\text{users} = \text{throughput} \times (\text{response time} + \text{think time})$$

. With 1,000 users, a 5 second response and 15 seconds of think time, the server must handle

$$\frac{1000}{20} = 50$$

requests per second.

Lesson: Load, stress, scalability, volume and soak

REMEMBER Security, accessibility and localization are covered in depth in the chapter "Security, Accessibility, Localization and Mobile Testing".

Lesson: The quality model: ISO/IEC 25010:2023

3.3 Security, Accessibility, Localization and Mobile Testing

A login page can work perfectly and still let a stranger read your bank statement. A shop can sell well and still lock out a blind customer. This page covers the checks that protect people: security, accessibility, other languages and countries, mobile phones, and privacy.

KEY IDEAS

- **Security testing: what it protects** Security testing checks that the application protects its data and its users from people who want to misuse it. A functional tester asks "does it work for the right user?" A security tester also asks "what happens when the wrong user tries?"
- **The OWASP Top 10 (2025) for testers** OWASP, the Open Worldwide Application Security Project, publishes a list of the ten most serious risks for web applications, based on data from thousands of real applications. The newest edition is the OWASP Top 10:2025 .
- **Accessibility: software for everyone** Accessibility testing checks that people with disabilities can use the product. That includes people who are blind or have low vision, are deaf, cannot use a mouse, have tremors, dyslexia or memory problems.
- **How to test accessibility by hand** Automated tools such as axe, Lighthouse and WAVE are a good first step, but they catch only part of the problems. They can see that an image has no alt text. They cannot tell that the alt text "image123" is useless. A manual pass is always needed.
- **Globalization: one product, many countries** When a product is sold in many countries, two jobs appear:
- **Mobile app testing** A phone is not a small computer. It moves between networks, runs on a battery, gets calls in the middle of a task, and comes in thousands of models. Mobile testing adds these questions to normal functional testing.
- **Privacy and data protection testing** Laws now give people rights over their personal data, and products must prove they respect them. Testers check these rights like any other requirement.

FORMULAS

How to test accessibility by hand

Exam favourite

$$\text{contrast ratio} = \frac{L_{light} + 0.05}{L_{dark} + 0.05}$$

Lesson: How to test accessibility by hand

TABLES

Table: rank · risk · what a manual tester can try

Rank	Risk	What a manual tester can try
A01	Broken Access Control	Change an id in the address or request and see if another user's data appears. Open an admin page as a normal user.
A02	Security Misconfiguration	Look for default passwords, detailed error pages with stack traces, open directory listings, test pages left in production.
A03	Software Supply Chain Failures (new)	Ask whether third party libraries are scanned and kept up to date. Check that the app loads scripts only from trusted places.
A04	Cryptographic Failures	Confirm HTTPS everywhere, and that passwords and card numbers never appear in URLs, logs, emails or screenshots.
A05	Injection	Type special characters such as a single quote,